



27 Feb 2026

Progress Report 1 on

**Power System Assessment of Dynamic Line Rating for Alberta
Transmission Utilities**

Course Name: Graduate Project in Electrical Engineering

Course Title: ENEL **678**

Submitted By,

Group No: **01**

Ahmed Mashook Iqbal

Project Manager

Email ID: ahmedmashook.iqbal@ucalgary.ca

UCID: **30267500**

Asif Rahman
UCID: **30248137**

Monjurul Haque
UCID: **30261730**

Eisa Kanhoush
UCID: **30273526**

G K Md Muttakin
UCID: **30246286**

Table of Contents

1. [Progress Made So Far](#)
2. [Methodology](#)
3. [Challenges Encountered](#)
4. [Roles & Responsibilities](#)
5. [List of Planned Work](#)
6. [Updated Risk Assessments](#)
7. [Budget Updates](#)
8. [Conclusion](#)
9. [Appendix](#)

Progress Made So Far

The following deliverables are directly referenced from the approved Design Roadmap document, and their completion percentages reflect progress as of February 27, 2026.

Project Deliverables:

1. *Design & construction of a physical DLR sensing prototype using an Arduino microcontroller with temperature, wind speed, and solar sensors. **Percentage of Completion: 100%***
Description: We have successfully designed and implemented the prototype as per the circuit diagram. The hardware system is fully assembled and calibrated. All sensors (TMP36, Light, Wind) are functional and tested. The updated circuit diagram and hardware prototype images are included in Appendix Sections 1 and 4.
2. *A Python/C++ implementation of the IEEE 738 Standard (Heat Balance Equation) to calculate the real-time ampacity of a conductor based on live sensor data. **Percentage of Completion: 100%***
Description: After preparing the physical prototype, we were able to send the data to Python and managed to calculate the ampacity of a conductor based on live sensor data. The scripting is done and it shows meaningful results. The mathematical model is fully coded in Python/C++. Conductor temperature is set as a static variable to ensure safety while validating environmental sensitivity. The Arduino firmware and Python IEEE 738 implementation are provided in Appendix Sections 5 and 6.
3. *Development of a PSSE/MATPOWER power-flow analysis workflow in which live Dynamic Line Rating data from the hardware prototype is imported via Python scripting to assess N-1 contingency performance in an Alberta transmission network. **Percentage of Completion: 30%***
Description: The Python bridge successfully pulls live serial data from the Arduino. The "Dynamic Push" logic for PSSE is currently in the active development stage.
4. *Experimental validation results demonstrating dynamic power flow response under varying environmental conditions, along with a final technical report. **Percentage of Completion: 10%***
Description: Benchmarking is complete. We have successfully collected and logged datasets for six environmental conditions: high/low temperature, high/low wind, and high/low light. The dynamic push and grid impact needs to be done on the next deliverables. We are yet to implement the findings on test grids or buses to fully achieve the objective of our project. Sample logged datasets are included in Appendix Section 7.

These completed tasks align with the Phase-1 roadmap schedule, and current PSSE integration remains on track with no major deviations from the planned timeline.

Methodology

1. **Sensor Characterization:** Individual sensors (TMP36, Anemometer, Light Sensor) were tested under controlled conditions to ensure accuracy and responsiveness within engineering tolerance.
2. **Signal Validation:** Arduino Serial output confirmed correct Analog-to-Digital Conversion (ADC) and data packet transmission.
3. **Multi-Sensor Integration:** All three sensors were integrated, and firmware confirmed no crosstalk or significant latency.

4. **Data Acquisition & Formatting:** A Python bridge pulled live Arduino data, structuring it into CSV format for future dynamic data injection into PSSE.
5. **Dynamic Ampacity Calculation:** The IEEE 738 heat balance equation was applied in Python to successfully calculate real-time DLR, demonstrating significant ampacity increases (Δ) over SLR.

Alignment with Design Roadmap: The methodology strictly followed the roadmap's staged approach: sensor validation, embedded integration, and dynamic power system modeling (Environmental Sensing → Embedded Processing → Data Structuring → Thermal Modeling → Dynamic Rating Output). This confirmed strong consistency between planning and execution.

Deviations and Justification: We have removed the LED display from the initial plan to keep the prototype simple and avoid unnecessary complexity.

Challenges Encountered

Challenge	Description	Impact	Mitigation
Technical	PSSE compatibility with latest Python version	Data transfer instability	Deployment of a stable legacy Python version compatible with PSSE API.
Logistical	Limited storage and safe handling of hardware prototype	Increased setup time and risk of accidental damage	Dedicated storage space has been secured.
Teamwork	Scheduling conflicts	Delayed planning discussions	Shared availability calendar and structured task delegation.

Roles & Responsibilities

Responsible Person	Roles & Responsibilities
Ahmed Mashook Iqbal	Designed and assembled hardware prototypes, calibrated TMP36, wind, and light sensors, implemented digital moving-average filtering, performed six-condition environmental validation testing.
Monjurul Haque	Developed Arduino-to-Python communication script, implemented IEEE 738 steady-state heat balance model, validated dynamic ampacity outputs.
G K Md Muttakin	Developed Python–PSSE API interface, tested serial data ingestion for dynamic branch rating updates, initiated single-bus dynamic rating validation.
Asif Rahman	Compiled environmental datasets, analyzed ampacity improvements over SLR, structured documentation and prepared progress report draft.
Eisa Kanhoush	Revised circuit diagram, optimized wiring layout for stability, implemented hardware refinements to reduce signal noise.

List of Planned Work for the Following Month

Planned Tasks	Deliverables	Timeline
Multi-bus PSSE integration	Dynamic rating injection across Alberta corridor model	March 10
N-1 contingency validation	Simulation-based stability assessment	March 17
Final system validation	Complete experimental grid response analysis	March 25

Updated Risk Assessments

Risk	Impact	Likelihood	Mitigation	Status
Electrical noise in analog temperature sensor	Fluctuating IEEE 738 ampacity values	Medium	Digital moving-average filtering implemented, wiring optimized	Partially mitigated
Sensor measurement inaccuracy	Incorrect DLR estimation and unrealistic ampacity margin	Low–Medium	Calibration under controlled conditions; validation against reference values	Controlled
PSSE data integration challenge	Simulation instability or incorrect branch updates	Medium	Asynchronous serial reading and sample-and-hold logic (>2% threshold)	Under implementation
Numerical non-convergence during N-1 simulation	Unrealistic contingency analysis results	Low	Dynamic rating capped at 30% above Static Line Rating	Mitigation implemented
Extreme environmental scenario overestimation	Overstated DLR improvement in simulations	Low–Medium	Use realistic operational bands and compare with SLR baseline	Identified & controlled

All identified risks are actively monitored, and mitigation effectiveness will be verified through hardware testing and PSSE validation during the next development phase.

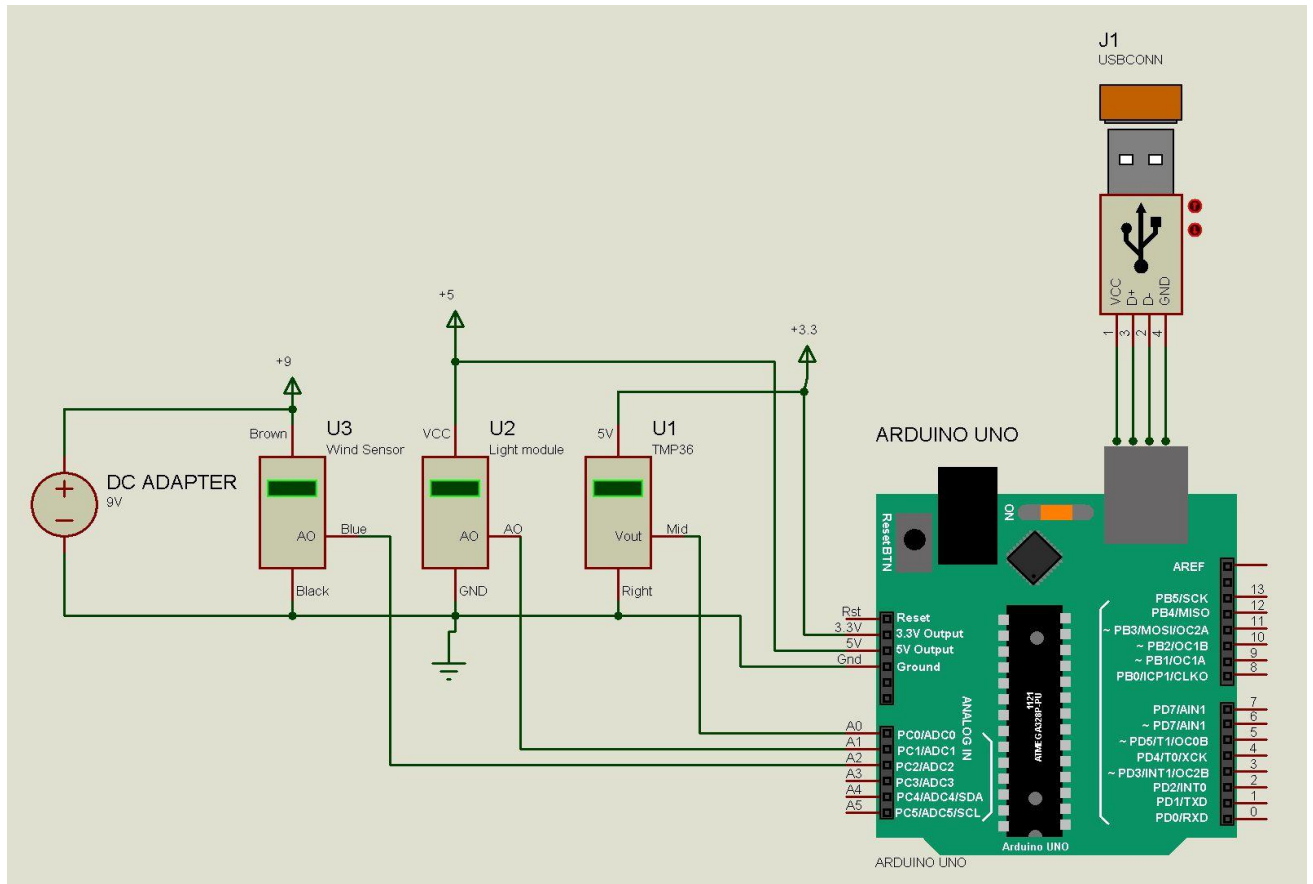
Budget Updates

The original estimated cost was 120 CAD $\pm$ 5% (114–126 CAD). The current total expenditure is 118 CAD, which remains within the approved budget range. No financial deviations have occurred.

Conclusion

The project remains on schedule and aligns with the Design Roadmap. The hardware prototype has been validated, and real-time Dynamic Line Rating (DLR) calculations based on the IEEE 738 heat balance model are operational in Python. PSSE integration is progressing with an active data bridge and simulation workflow under refinement. Experimental testing confirms measurable ampacity improvement over Static Line Rating. The remaining technical challenge is ensuring numerical convergence during full N-1 contingency simulations with dynamic branch updates, which will be resolved in the next phase. Final efforts will focus on complete PSSE integration and system-level validation to achieve full project objectives.

Appendix



1.

Fig: Updated circuit diagram

sketch_feb22a | Arduino IDE 2.3.7

File Edit Sketch Tools Help

Arduino UNO

sketch_feb22a.ino

```

34   if (windSpeed < 0) windSpeed = 0;
35
36   // ----- Print All Data -----
37   Serial.print("TEMP_C=");
38   Serial.print(tempC, 1);
39
40   Serial.print(" | LIGHT%=");
41   Serial.print(lightPercent, 1);
42

```

Output Serial Monitor X

Message (Enter to send message to 'Arduino UNO' on 'COM4')

New Line 9600 baud

```

TEMP_C=20.9 | LIGHT%=27.5 | WIND_V=0.391 V | WIND=0.00 m/s
TEMP_C=20.9 | LIGHT%=27.8 | WIND_V=0.381 V | WIND=0.00 m/s
TEMP_C=21.4 | LIGHT%=27.9 | WIND_V=0.386 V | WIND=0.00 m/s
TEMP_C=21.8 | LIGHT%=26.8 | WIND_V=0.479 V | WIND=1.60 m/s
TEMP_C=21.8 | LIGHT%=26.8 | WIND_V=0.474 V | WIND=1.50 m/s
TEMP_C=20.9 | LIGHT%=26.6 | WIND_V=0.474 V | WIND=1.50 m/s
TEMP_C=20.9 | LIGHT%=26.9 | WIND_V=0.445 V | WIND=0.91 m/s
TEMP_C=21.4 | LIGHT%=27.0 | WIND_V=0.606 V | WIND=4.17 m/s
TEMP_C=21.4 | LIGHT%=27.1 | WIND_V=0.567 V | WIND=3.38 m/s
TEMP_C=21.8 | LIGHT%=27.0 | WIND_V=0.547 V | WIND=2.99 m/s

```

Ln 44, Col 32 Arduino UNO on COM4

2.

```

RAW: 22.83,54.94,0.508,2.193
2026-02-24T17:34:01 T=22.8°C Light=54.9% Wind=2.19 m/s G=549 W/m² Dyn=1898.4 A Static=357 A +Δ=1541.4 A
RAW: 22.83,54.55,0.503,2.094
2026-02-24T17:34:01 T=22.8°C Light=54.5% Wind=2.09 m/s G=546 W/m² Dyn=1855.3 A Static=357 A +Δ=1498.3 A
RAW: 23.31,54.74,0.508,2.193
2026-02-24T17:34:02 T=23.3°C Light=54.7% Wind=2.19 m/s G=547 W/m² Dyn=1889.6 A Static=357 A +Δ=1532.6 A
RAW: 22.34,54.74,0.508,2.193
2026-02-24T17:34:02 T=22.3°C Light=54.7% Wind=2.19 m/s G=547 W/m² Dyn=1907.3 A Static=357 A +Δ=1550.3 A
RAW: 22.34,54.45,0.503,2.094
2026-02-24T17:34:03 T=22.3°C Light=54.5% Wind=2.09 m/s G=544 W/m² Dyn=1864.0 A Static=357 A +Δ=1507.0 A
RAW: 22.83,54.55,0.508,2.193
2026-02-24T17:34:03 T=22.8°C Light=54.5% Wind=2.19 m/s G=546 W/m² Dyn=1898.4 A Static=357 A +Δ=1541.4 A
RAW: 22.83,54.35,0.508,2.193
2026-02-24T17:34:04 T=22.8°C Light=54.4% Wind=2.19 m/s G=544 W/m² Dyn=1898.4 A Static=357 A +Δ=1541.4 A
RAW: 22.83,54.45,0.503,2.094
2026-02-24T17:34:04 T=22.8°C Light=54.5% Wind=2.09 m/s G=544 W/m² Dyn=1855.3 A Static=357 A +Δ=1498.3 A

```

3.

Sensor Data Readings


```

const int WIND_PIN = A2;

void setup() {
  Serial.begin(9600);

  // Sending Header
  Serial.println("temp_c,light_percent,wind_voltage,wind_ms");
}

void loop() {
  // ---- TMP36 ----
  int rawTmp = analogRead(TMP_PIN);
  float tmpVoltage = rawTmp * (5.0 / 1023.0); // ADC ref = 5V
  float tempC = (tmpVoltage - 0.5) * 100.0; // TMP36 formula

  // ---- Light sensor ----
  int rawLight = analogRead(LIGHT_PIN);
  float lightPercent = (rawLight / 1023.0) * 100.0;

  // ---- Wind sensor ----
  int rawWind = analogRead(WIND_PIN);
  float windVoltage = rawWind * (5.0 / 1023.0);

  // Map 0.40V -> 0 m/s, 2.00V -> 32.4 m/s
  float windSpeed = (windVoltage - 0.40) * (32.4 / (2.0 - 0.4));
  if (windSpeed < 0) windSpeed = 0;

  // ---- CSV output ----
  Serial.print(tempC, 2);
  Serial.print(",");
  Serial.print(lightPercent, 2);

```

```

Serial.print(",");
Serial.print(windVoltage, 3);
Serial.print(",");
Serial.println(windSpeed, 3);

delay(500); // log every 0.5 s
}

```

6. *Arduino to Python Script:*

```

import serial
import csv
from datetime import datetime
import math

# ----- SERIAL SETTINGS -----
# Change this to match your Arduino port (check Arduino IDE → Tools → Port)
PORT = "COM4"
BAUD = 9600

# ----- DLR PARAMETERS: ACSR 4/0 PENGUIN -----
# Data from ACSR 4/0 "PENGUIN" datasheets (primary/secondary
# distribution conductor in overhead systems):
# - Diameter: 0.563 in -> 0.0143 m
# - DC resistance @ 20°C: 0.0795 ohm / 1000 ft -> ~0.261 ohm/km
# - Static ampacity (nameplate): 357 A

D_m = 0.0143    # conductor outer diameter [m]
R20_km = 0.261  # DC resistance at 20°C [ohm/km]
alpha_R = 0.00403 # temp coeff of resistance [1/°C] (aluminium)
emissivity = 0.5  # weathered surface emissivity
alpha_solar = 0.5 # solar absorptivity
Tc_max = 75.0    # allowed conductor temperature [°C] (use 80.0 if you prefer)

```

STATIC_AMPACITY = 357.0 # A, datasheet static rating for Penguin

Light% -> solar irradiance [W/m^2] (very rough model)

100% light → ~1000 W/m^2

K_SOLAR = 10.0 # W/m^2 per % light

Simple convection factor [W/(m·K) per m/s] (tunable)

K_CONV = 10.0 # adjust later if needed

----- DLR / AMPACITY FUNCTION -----

def compute_ampacity(Ta_C, wind_ms, G_solar_Wm2):

"""

Compute a simplified dynamic ampacity [A]

using a steady-state heat balance:

$$I^2 * R + q_s = q_c + q_r$$

Ta_C : ambient temperature [°C]

wind_ms : wind speed [m/s]

G_solar_Wm2 : solar irradiance [W/m^2]

"""

Convert R20 from ohm/km -> ohm/m

R20_m = R20_km / 1000.0

Conductor resistance at Tc_max

R_T = R20_m * (1.0 + alpha_R * (Tc_max - 20.0)) # ohm/m

---- Convective heat loss qc [W/m] (simple model) ----

$q_c \approx k_{conv} * v * (T_c - T_a)$

if wind_ms < 0:

wind_ms = 0.0

qc = K_CONV * wind_ms * (Tc_max - Ta_C)

```

# ---- Radiative heat loss qr [W/m] ----
sigma = 5.670374419e-8 # W/m^2/K^4
Tc_K = Tc_max + 273.15
Ta_K = Ta_C + 273.15
qr = sigma * emissivity * math.pi * D_m * (Tc_K**4 - Ta_K**4)

# ---- Solar heat gain qs [W/m] ----
# projected area per meter ≈ D_m [m^2/m] for a long cylinder
qs = alpha_solar * G_solar_Wm2 * D_m

# Net cooling available for Joule heating I^2 R
net_cooling = qc + qr - qs

if net_cooling <= 0:
    # Environment is heating more than cooling; no margin for current
    return 0.0

# I^2 * R = net_cooling -> I = sqrt(net_cooling / R)
I_max = math.sqrt(net_cooling / R_T)
return I_max

# ----- MAIN: READ ARDUINO + COMPUTE DLR -----
def main():
    ser = serial.Serial(PORT, BAUD, timeout=1)

    # CSV log file
    filename = f"dln_log_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
    f = open(filename, "w", newline="")
    writer = csv.writer(f)
    writer.writerow([

```

```

        "timestamp",
        "temp_c",
        "light_percent",
        "wind_voltage",
        "wind_ms",
        "G_solar_Wm2",
        "dynamic_ampacity_A",
        "static_ampacity_A",
        "extra_margin_A"
    ])

    print(f"Listening on {PORT} at {BAUD} baud...")
    print(f"Logging to {filename}")
    print("Press Ctrl+C to stop.\n")

    try:
        while True:
            line = ser.readline().decode("utf-8").strip()
            if not line:
                continue

            # Show raw line for debugging
            print("RAW:", line)

            # Skip the header line from Arduino
            if line.startswith("temp_c"):
                continue

            parts = line.split(",")
            if len(parts) != 4:
                print("Skipped malformed line")
                continue

```

```

# Arduino CSV: temp_c,light_percent,wind_voltage,wind_ms
try:
    temp_c    = float(parts[0])
    light_percent = float(parts[1])
    wind_voltage = float(parts[2]) # not used in DLR, but logged
    wind_ms    = float(parts[3])
except ValueError:
    print("Skipped non-numeric line")
    continue

# Convert light % to approximate solar irradiance [W/m^2]
G_solar = max(light_percent, 0.0) * K_SOLAR

# Compute dynamic ampacity
dyn_ampacity = compute_ampacity(temp_c, wind_ms, G_solar)

# Compare vs static rating
extra_margin = max(dyn_ampacity - STATIC_AMPACITY, 0.0)

# Timestamp
ts = datetime.now().isoformat(timespec="seconds")

# Log everything
writer.writerow([
    ts,
    temp_c,
    light_percent,
    wind_voltage,
    wind_ms,
    G_solar,
    dyn_ampacity,

```

```

        STATIC_AMPACITY,
        extra_margin
    ])

    # Print a clean summary INCLUDING LIGHT %
    print(
        f"{ts} T={temp_c:.1f}°C Light={light_percent:.1f}% "
        f"Wind={wind_ms:.2f} m/s G={G_solar:.0f} W/m² "
        f"Dyn={dyn_ampacity:.1f} A Static={STATIC_AMPACITY:.0f} A "
        f"+Δ={extra_margin:.1f} A"
    )

except KeyboardInterrupt:
    print("\nStopped by user.")
finally:
    f.close()
    ser.close()
    print("Serial closed, file saved.")

if __name__ == "__main__":
    main()

```

7. **Data log** (some data are given)

Temp c	Light (%)	Wind volt	Wind m/s	Solar W/m ²	DLR	SLR	Difference
22.83	59.24	0.411	0.214	592.4	604.06	357	247.06
23.31	59.34	0.411	0.214	593.4	601.21	357	244.21
22.34	59.34	0.425	0.511	593.4	926.75	357	569.75
22.83	59.34	0.415	0.313	593.4	725.85	357	368.85
22.34	59.24	0.425	0.511	592.4	926.77	357	569.77
22.34	59.24	0.42	0.412	592.4	833.90	357	476.90
22.34	59.24	0.42	0.412	592.4	833.90	357	476.90
22.83	59.04	0.425	0.511	590.4	922.43	357	565.43

Temp c	Light (%)	Wind volt	Wind m/s	Solar W/m ²	DLR	SLR	Difference
22.83	59.63	0.42	0.412	596.3	829.92	357	472.92
23.31	59.53	0.435	0.709	595.3	1078.83	357	721.83
22.34	59.24	0.435	0.709	592.4	1088.99	357	731.99
23.31	59.24	0.43	0.61	592.4	1001.72	357	644.72
22.34	59.24	0.435	0.709	592.4	1088.99	357	731.99
21.85	58.94	0.43	0.61	589.4	1015.89	357	658.89
23.8	58.55	0.43	0.61	585.5	997.00	357	640.00
23.31	58.55	0.43	0.61	585.5	1001.80	357	644.80
21.85	58.65	0.43	0.61	586.5	1015.93	357	658.93
22.83	58.55	0.43	0.61	585.5	1006.47	357	649.47
23.31	58.75	0.44	0.808	587.5	1150.88	357	793.88
22.34	58.85	0.435	0.709	588.5	1089.03	357	732.03
22.34	54.06	0.44	0.808	540.6	1162.14	357	805.14
23.8	48.97	0.445	0.907	489.7	1213.70	357	856.70
23.31	50.54	0.435	0.709	505.4	1079.76	357	722.76
23.31	49.66	0.435	0.709	496.6	1079.85	357	722.85
22.34	48.39	0.435	0.709	483.9	1090.11	357	733.11
23.31	47.9	0.43	0.61	479	1002.99	357	645.99
23.31	48.29	0.44	0.808	482.9	1151.90	357	794.90
22.83	48.19	0.44	0.808	481.9	1157.27	357	800.27
22.83	48.09	0.435	0.709	480.9	1085.04	357	728.04
23.31	48.39	0.445	0.907	483.9	1219.57	357	862.57
22.83	49.17	0.445	0.907	491.7	1225.16	357	868.16
22.83	49.27	0.45	1.006	492.7	1289.57	357	932.57
22.83	49.46	0.445	0.907	494.6	1225.14	357	868.14
22.83	49.36	0.45	1.006	493.6	1289.56	357	932.56
22.83	48.88	0.455	1.105	488.8	1350.95	357	993.95
22.34	48.97	0.44	0.808	489.7	1162.63	357	805.63
22.34	48.97	0.445	0.907	489.7	1230.94	357	873.94
22.83	48.39	0.45	1.006	483.9	1289.65	357	932.65
22.34	48.68	0.45	1.006	486.8	1295.68	357	938.68
23.31	48.88	0.45	1.006	488.8	1283.64	357	926.64
22.83	49.07	0.45	1.006	490.7	1289.59	357	932.59
23.8	49.17	0.445	0.907	491.7	1213.69	357	856.69
23.8	32.45	0.445	0.907	324.5	1215.23	357	858.23
22.83	30.69	0.445	0.907	306.9	1226.85	357	869.85